

Reformat Project Task List

Module_Scanner Audit

- ☐ 1) Verify if the app properly locks out user inputs while automation is occurring.
- ☐ 2) Check if tests exist to validate this input lockout functionality.

Core Feature Removals

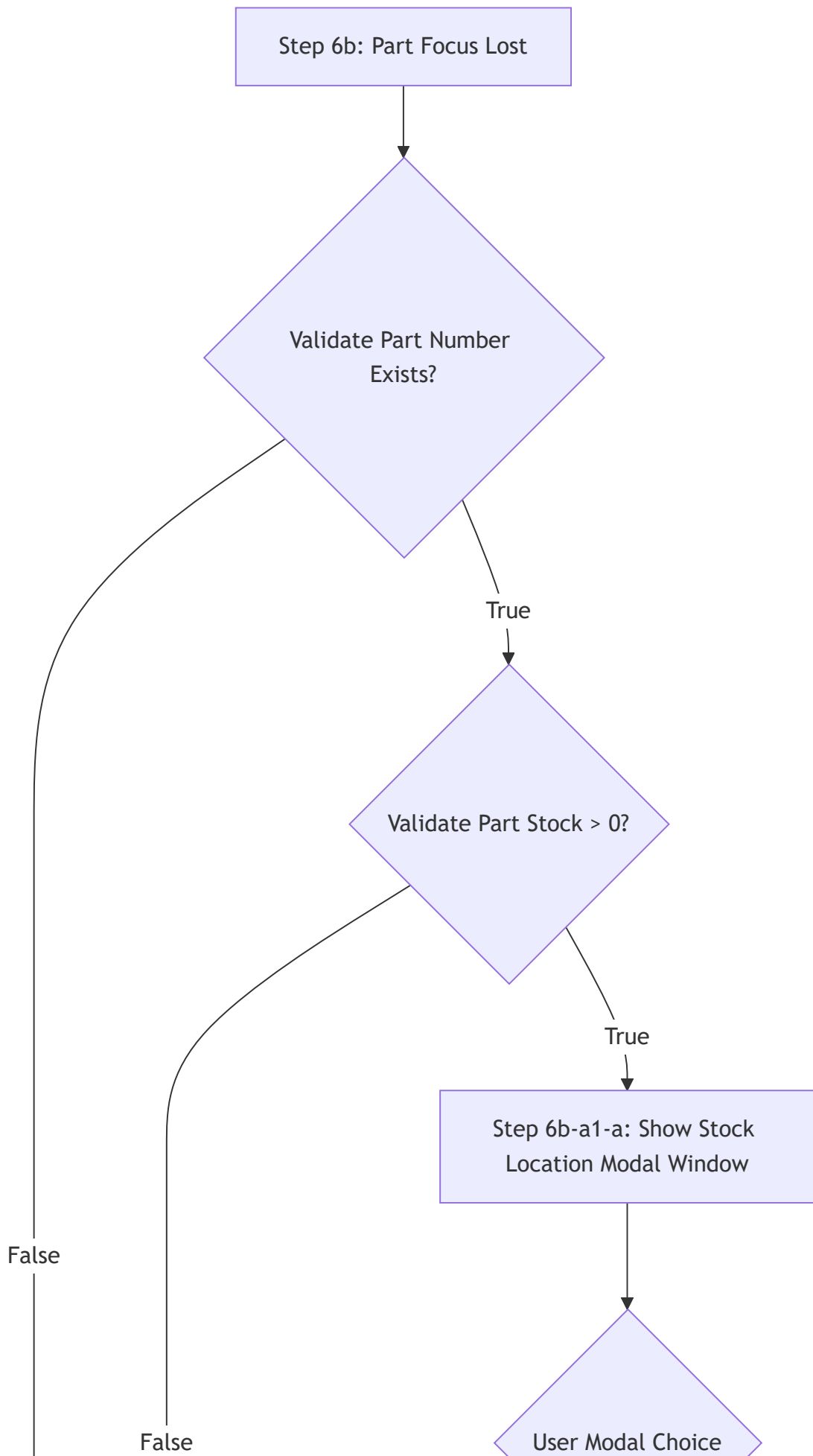
- ☐ 3) Remove "Save Draft" button and all underlying functionality.
- ☐ 4) Remove "Save for Later" button and all underlying functionality.

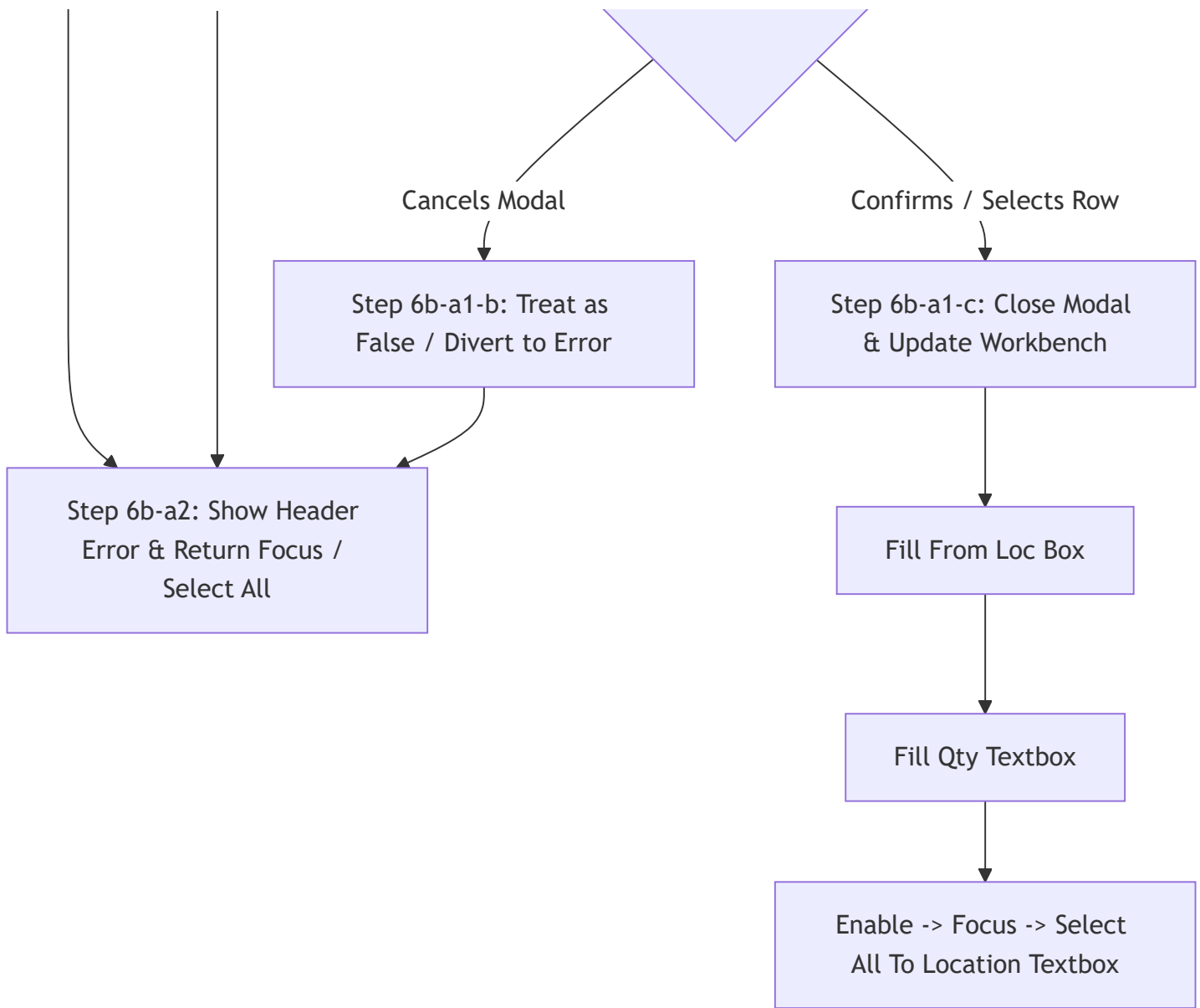
UI & Layout Adjustments

- ☐ 5) Fix "Manage Item" modal window size (content is currently cut off on the UI).
- ☐ 6) Update Workbench inputs to the following baseline state:
 - **Part Number:** Editable.
 - **From Location:** Read-Only (at all times).
 - **To Location:** Read-Only (at all times) and Disabled.
 - **Quantity:** Read-Only (at all times).
 - **Add Button:** Disabled.

Part Number Validation Workflow (Step 6b)

Workflow Diagram





[Step 6b] Part Focus Lost & Validation

- When the user enters a **Part Number** and focus is lost, validate if the part number exists.

[Step 6b-a1] Case A: Part Number Exists & Has Stock

- **[Step 6b-a1-a] Display Modal Window:** Show a modal listing all current locations holding > 0 on hand.
 - Each row must include a **Quantity Textbox** auto-filled with the full quantity of that location.
 - Allow editing of this textbox (enforce boundaries: cannot go below 0 or above the location's maximum quantity).
- **[Step 6b-a1-b] User Cancels Modal:** Treat as if the part number had no stock. Divert immediately to **[Step 6b-a2]**.
- **[Step 6b-a1-c] Close Modal / Confirm Action:** Perform the following automatic updates back on the main Workbench view:

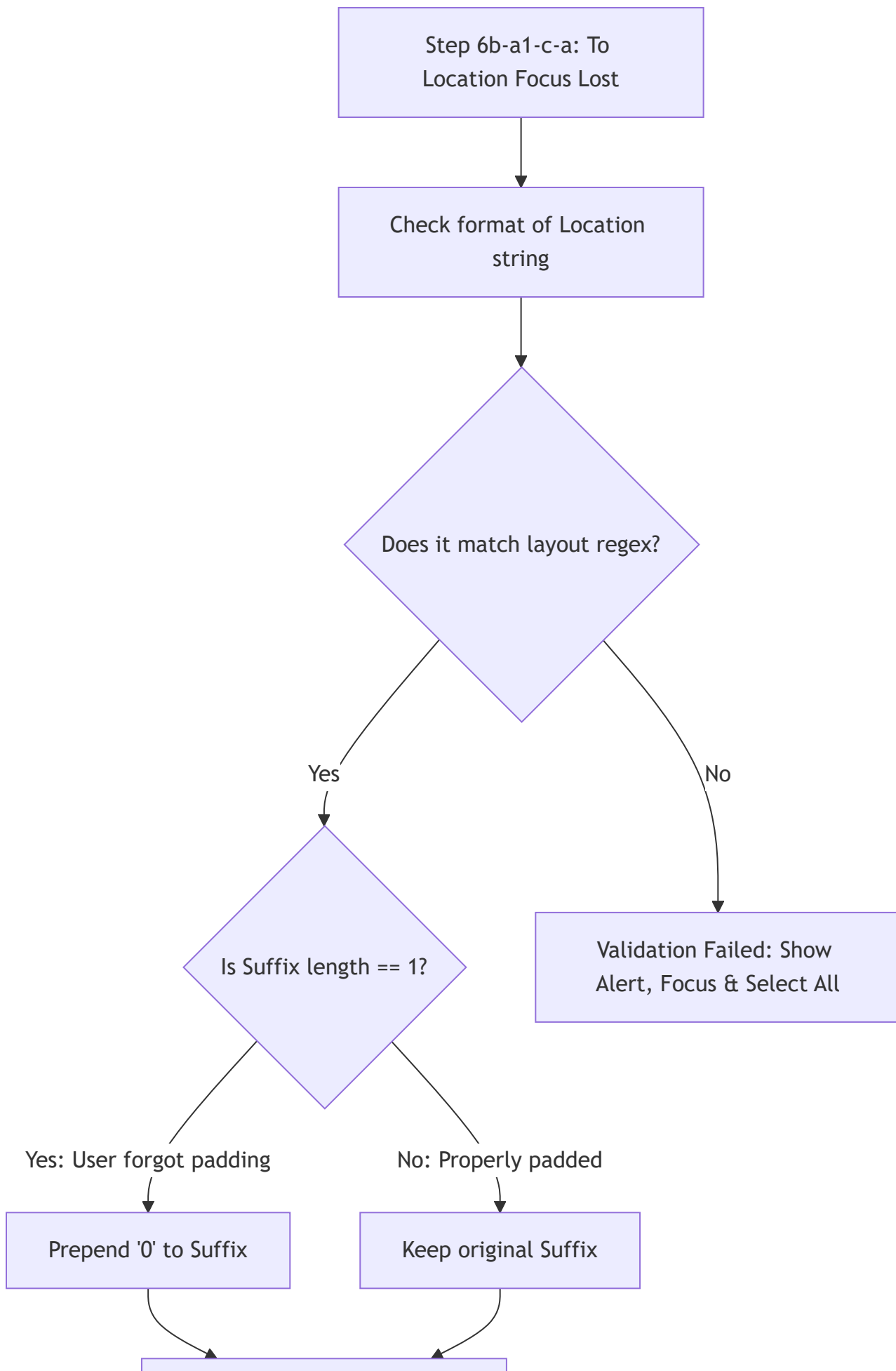
- i. Fill **From Loc** box with the location selected by the user.
- ii. Fill **Qty** textbox with the quantity value from the selected row.
- iii. Enable -> Focus -> Select All on the **To Location** textbox.

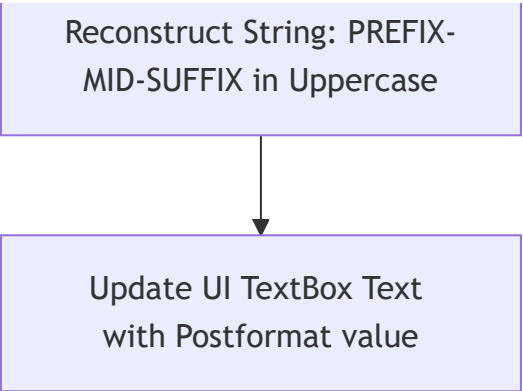
[Step 6b-a2] Case B: Part Number Has No Stock / Invalid

1. Show an inline error message in the **MainForm's Header panel** stating that the part number does not have any quantity in-house.
2. Return focus to the **Part Textbox** and execute a **Select All** command.

To Location Focus Lost Workflow (Step 6b-a1-c-a)

Formatting Diagram





- **[Step 6b-a1-c-a] Format Verification:** When focus is lost on the **To Location** textbox, parse and check the format of the location string.
- **[Step 6b-a1-c-b] Sanitation Logic:** Use the following reference tables to automatically sanitize and fix improper user formatting layouts:

Standard 2-Digit Padded Group

User Variant	Example User Input (Preformat)	Should Return (Postformat)
User Variant <i>The structural pattern of letters (L), numbers (N), and hyphens typed by the user.</i>	Example User Input (Preformat) <i>A real-world example of raw, unformatted text entered into the location field.</i>	Should Return (Postformat) <i>The final, cleaned up, and standardized string saved to the database.</i>
L-LN-NN	V-A0-01	V-A0-01
LL-LN-NN	VA-A0-01	VA-A0-01
LL-LNN	VA-A001	VA-A0-01
L-LNN	V-A001	V-A0-01
LLN-NN	VAA0-01	VA-A0-01
LN-NN	VA0-01	V-A0-01
LLNNN	VAA001	VA-A0-01
LNNN	VA001	V-A0-01

Unpadded Group (Auto-Corrected & Padded)

User Variant	Example User Input (Preformat)	Should Return (Postformat)
User Variant <i>The structural pattern where the user forgot to add a leading zero to the final section.</i>	Example User Input (Preformat) <i>The raw input string containing a single trailing digit instead of two.</i>	Should Return (Postformat) <i>The normalized string with the missing zero automatically added and hyphens inserted.</i>
L-LN-N	V-A0-1	V-A0-01
LL-LN-N	VA-A0-1	VA-A0-01
LL-LN	VA-A01	VA-A0-01
L-LN	V-A01	V-A0-01
LLN-N	VAA0-1	VA-A0-01
LN-N	VA0-1	V-A0-01
LLNN	VAA01	VA-A0-01
LNN	VA01	V-A0-01

Implementation Reference (C# Parsing Engine)

Place this sanitization logic inside your validation helper class to resolve all 16 layout variations documented above:

```

using System;
using System.Text.RegularExpressions;

public static class LocationParser
{
    private static readonly Regex LocationRegex = new Regex(
        @"^(?<prefix>[A-Za-z]{1,2})-?(?<mid>[A-Za-z][0-9])-?(?<suffix>[0-9]{1,2})\$",
        RegexOptions.Compiled
    );

    public static string SanitizeLocationFormat(string rawInput)
    {
        if (string.IsNullOrEmpty(rawInput)) return null;

        string input = rawInput.Trim();
        Match match = LocationRegex.Match(input);

        if (!match.Success) return null;

        string prefix = match.Groups["prefix"].Value.ToUpper();
        string mid     = match.Groups["mid"].Value.ToUpper();
        string suffix = match.Groups["suffix"].Value;

        if (suffix.Length == 1)
        {
            suffix = "0" + suffix;
        }

        return $"{prefix}-{mid}-{suffix}";
    }
}

```

UI Integration Reference (C# Control Events)

Bind this event handler to your **To Location** UI component to complete the focus lost requirement:

```
private void ToLocationTextBox_Leave(object sender, EventArgs e)
{
    TextBox toLocBox = (TextBox)sender;
    string rawValue = toLocBox.Text;

    string sanitizedValue = LocationParser.SanitizeLocationFormat(rawValue);

    if (sanitizedValue != null)
    {
        // Formatting succeeded: update UI box with Postformat value
        toLocBox.Text = sanitizedValue;
    }
    else
    {
        // Formatting failed: Alert user, re-focus, and highlight invalid entry
        MessageBox.Show("Invalid location layout entered. Please check your format.",
                        "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        toLocBox.Focus();
        toLocBox.SelectAll();
    }
}
```